
KernelGenBench: A Multi-Source and Multi-Chip Benchmark for LLM-based Kernel Generation

Peiyu Zang^{1,5}, Jian Tao⁵, Jialing Zhang^{2,5}, Yichen Yuan^{3,5},
Wentao Zhang⁴, Guang Liu^{5*}, Yonghua Lin⁵

¹Beijing Normal University, ²Beijing Jiaotong University,

³Institute of Automation, CAS, ⁴Peking University, ⁵BAAI

{202421130105@mail.bnu.edu.cn, liuguang@baai.ac.cn}

Abstract

Large language models (LLMs) have significantly increased the demand for efficient accelerator kernels, but kernel development remains a highly specialized and labor-intensive task. The recent rise of LLMs and agentic frameworks offers a promising pathway toward automatic kernel generation. However, despite rapid progress, there is still no comprehensive benchmark to rigorously evaluate LLM-generated kernels across diverse operator sources or heterogeneous hardware platforms. We present **KernelGenBench**², a unified benchmark for systematically evaluating LLM- and agent-generated Triton kernels across diverse operator sources and heterogeneous hardware platforms. It comprises two complementary sub-benchmarks: **KernelGenBench-MS (Multi-Source)**, evaluating 210 operators from three sources beyond standard PyTorch-centric tasks, and **KernelGenBench-MC (Multi-Chip)**, measuring performance portability across five alternative hardware platforms using a 110-operator subset. Our large-scale evaluation, consuming over 15 billion tokens, shows: (1) agent-based methods consistently outperform pure LLM sampling methods, while cuBLAS operators are the most challenging across all methods; (2) generation performance varies significantly across hardware platforms, with even state-of-the-art kernel-specialized agents experiencing severe cross-platform degradation (e.g., AutoKernel drops from 87% on NVIDIA to 25% on Platform E); (3) autonomous kernel generation remains highly cost-intensive, with specialized agent methods averaging 5.11 million tokens per successful operator (AKO4all reaches 5.19 million), orders of magnitude higher than simple LLM sampling approaches.

1 Introduction

The rapid advancement of large language models (LLMs) has significantly increased the demand for efficient accelerator kernels that are critical for high-performance model execution. However, designing such kernels remains a challenging task that requires deep expertise in low-level programming, hardware architectures, and performance optimization. Recently, the emergence of LLMs and agentic frameworks has created new opportunities for automatically generating GPU kernels. Existing work has primarily focused on NVIDIA-centric CUDA and Triton kernels [1], which continue to dominate current research and deployment settings. These advances raise the possibility of democratizing a process that has traditionally required significant domain expertise.

However, despite rapid progress in LLM-based kernel generation, there is still no comprehensive benchmark to rigorously evaluate their effectiveness and real-world utility. Existing high-level code

*Corresponding author.

²Code and data are publicly available at <https://github.com/flagos-ai/KernelGenBench>.

generation benchmarks [2, 3] focus primarily on functional correctness, while low-level kernels additionally require hardware efficiency [4, 5]. Meanwhile, existing kernel benchmarks [6, 7] remain narrowly scoped, typically focusing on single-source operators within restricted ecosystems, and fail to capture the complexity of real-world deployment scenarios across diverse operator sources and heterogeneous hardware platforms.

To address this gap, we introduce **KernelGenBench**, a multi-source and multi-chip benchmark that systematically evaluates LLM- and agent-generated Triton kernels across diverse operator sources and heterogeneous hardware platforms, ensuring cross-platform portability beyond traditional CUDA lock-in [8–10]. KernelGenBench consists of two complementary sub-benchmarks:

- **KernelGenBench-MS (Multi-Source):** Evaluates kernel generation across 210 operators spanning PyTorch ATen [11], vLLM [12], and cuBLAS baselines.
- **KernelGenBench-MC (Multi-Chip):** Assesses performance portability across six heterogeneous hardware platforms using a 110-operator ATen subset.

To enable robust and realistic evaluation, we design a production-aligned execution framework with a distributed sandbox infrastructure and a three-tier anti-hack mechanism that prevents benchmark evasion. We also introduce a multi-dimensional evaluation protocol that jointly measures functional correctness, execution speedup, and agentic cost efficiency.

We conducted extensive experiments consuming over 15 billion tokens. Our experiments show several key findings. (1) Agent-based methods consistently outperform pure LLM sampling approaches, but operator source imposes a clear difficulty hierarchy: ATen operators are the most tractable, vLLM operators are the hardest to generate correctly but offer the greatest speedup potential, and cuBLAS operators remain the most challenging overall due to severe optimization ceilings across all methods. (2) Generation performance varies substantially across hardware platforms, exposing major portability challenges, with even state-of-the-art kernel-specialized agents experiencing severe cross-platform degradation (e.g., AutoKernel drops from 87% on NVIDIA to 25% on Platform E). (3) Although autonomous agentic frameworks improve correctness through iterative debugging and optimization, they remain highly cost-intensive, with specialized methods averaging 5.11 million tokens per successful operator (AKO4all reaches 5.19 million), orders of magnitude higher than simple LLM sampling approaches.

2 Related Work

2.1 LLM-Driven Kernel Generation

Recent work on automated accelerator kernel generation spans two tracks [13]. On the model side, approaches include supervised fine-tuning on compiler-generated datasets [14], reinforcement learning with compilation feedback [15], and execution-grounded post-training on industrial workloads [16]. However, static inference restricts optimization to a single pass, motivating iterative agentic systems that autonomously debug and optimize Triton code via execution-driven reinforcement [17], evolutionary search [18], and runtime log exploitation [19]. While these systems have proven effective at achieving functional correctness, their rapid advancement shifts the bottleneck to the urgent need for rigorous, large-scale evaluation benchmarks.

2.2 Limitations of Existing Benchmarks

As kernel agents rapidly mature, existing benchmarks struggle to adequately evaluate their boundaries, particularly in cross-platform scenarios and multi-source operator generation. We categorize current evaluation limitations along these two dimensions:

Single-source constraints. KernelBench [6] and TritonBench [7] pioneered execution-based evaluation, but their exclusive focus on standardized PyTorch primitives has allowed kernel-specialized agents to reach near 100% correctness, creating an illusion of solvedness [20]. BackendBench [21] integrates Triton into PyTorch but still operates at a high abstraction level, lacking the unstructured engineering complexity of production-grade libraries like vLLM or cuBLAS. Furthermore, inspired by SWE-bench [22] in general software engineering, we argue that kernel evaluation must move beyond single-pass correctness to assess closed-loop agentic debugging and its associated token costs.

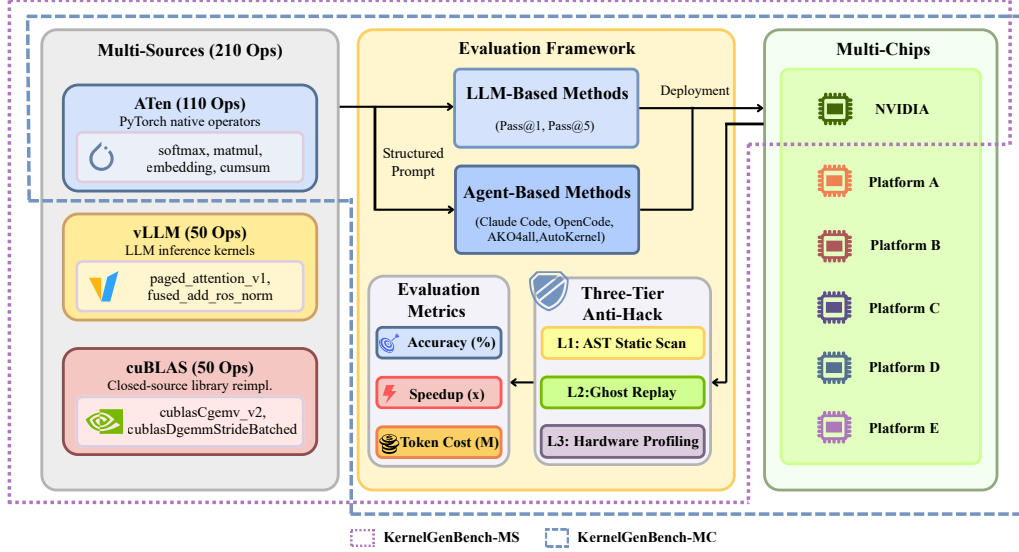


Figure 1: Overview of KernelGenBench. The purple and blue dashed boxes represent the two sub-benchmarks, which share the central evaluation infrastructure.

Single-hardware lock-in. The vast majority of kernel benchmarks remain strictly confined to the NVIDIA ecosystem. FlashInfer-Bench [23] and SOL-ExecBench [24] introduce valuable performance metrics but target only single-GPU NVIDIA settings. While MultiKernelBench [25] makes a preliminary attempt at cross-platform evaluation, it lacks a unified verification pipeline and consistent execution baselines. Consequently, no existing benchmark has systematically measured the severe performance portability gap—the divergence in execution speedup caused by fragmented compiler maturities across heterogeneous hardware platforms.

3 KernelGenBench

We introduce **KernelGenBench**, a unified benchmark comprising two complementary sub-benchmarks: KernelGenBench-MS and KernelGenBench-MC, as illustrated in Figure 1.

3.1 KernelGenBench-MS

To measure the true boundaries of kernel generation capabilities, KernelGenBench-MS is anchored on a stable NVIDIA hardware baseline (controlling for compiler maturity) and curates 210 operators from three complementary sources, each representing a distinct category of real-world kernel workloads: framework-level open-source operators (PyTorch ATen), CUDA open-source library operators (vLLM), and CUDA closed-source library operators (cuBLAS). A key design principle is that we explicitly provide detailed API signatures for all operators, and the generated Triton kernels must exactly match these specifications, enabling them to serve as drop-in replacements for production [26]. All prompt contexts are constructed by HPC engineers following actual development workflows (Appendix A). Full operator lists are in Appendix B.

Source 1: PyTorch ATen Operators (110 problems). We systematically filter out of the 900+ `torch.ops.aten` APIs: the top-50 most frequent operators are extracted from real training traces of 2,907 open-source models, and 60 long-tail operators are uniformly sampled. Each operator API is an independent problem (e.g., `torch.abs` and `torch.abs_` are two separate problems), requiring the model to implement all overload variants. Prompts are constructed by dynamically extracting the `FunctionSchema` and official docstrings. The baseline directly invokes native ATen APIs.

Source 2: vLLM Operators (50 problems). We select operators with standalone CUDA implementations from vLLM, covering attention mechanisms (e.g., `paged_attention_v1`), KV cache

management and mixed-precision quantization (FP8/AWQ).³ This subset tests whether models can generate the acceleration kernels used in real-world LLM inference.

Source 3: cuBLAS Operators (50 problems). We designate the cuBLAS closed-source dynamic library as the absolute performance baseline.⁴ Critically, the baseline loads `libcublas.so` directly via `ctypes.cdll`, bypassing all high-level wrappers. To construct a balanced problem set, we first identify the 10 most frequently invoked cuBLAS routines via profiling traces and expand them across precision and batching modes. Furthermore, to prevent the evaluation from over-indexing on a single operator family (e.g., where mastering one GEMM variant allows trivial generalization to all others), we strategically sample standard versions of other functionally diverse BLAS routines. This compels the model to match proprietary performance across a broad spectrum of linear algebra tasks without relying on external libraries. Table 1 illustrates the fine-grained API fragmentation using GEMM as an example.

Table 1: Fine-grained cuBLAS operator variants: GEMM (General Matrix Multiplication) as an example. 14 independent problems are derived from a single operator family across precision, batching mode, and index width.

Precision	Standard (.v2)	StridedBatched	Batched	64-bit Index (.64)	Other Variants
Float32 (S)	<code>cublasSgemm.v2</code>	<code>cublasSgemmStridedBatched</code>	<code>cublasSgemmBatched.64</code>	—	<code>cublasSgemmEx</code>
Float64 (D)	—	<code>cublasDgemmStridedBatched</code>	<code>cublasDgemmBatched</code>	<code>cublasDgemmStridedBatched.64</code>	—
Complex64 (C)	<code>cublasCgemm.v2</code>	<code>cublasCgemmStridedBatched</code>	—	<code>cublasCgemmStridedBatched.64</code>	—
Complex128 (Z)	—	<code>cublasZgemmStridedBatched</code>	<code>cublasZgemmBatched</code>	—	—
Float16 (H)	—	<code>cublasHgemmStridedBatched</code>	<code>cublasHgemmBatched</code>	—	—

3.2 KernelGenBench-MC

To systematically evaluate models’ kernel generation capabilities across heterogeneous hardware platforms, KernelGenBench-MC establishes a unified evaluation framework. To control for task complexity, this sub-benchmark relies exclusively on the ATen operator subset.

The framework natively targets six distinct architectures: NVIDIA A100 and five alternative hardware platforms (Platform A–E), whose identities are anonymized at their request due to confidentiality agreements. These platforms represent diverse architectural paradigms and ecosystem maturity levels: CUDA-compatible architectures with established toolchains, proprietary instruction sets with vendor-specific software stacks, and emerging accelerators with nascent compiler infrastructures. Rather than testing in isolation, KernelGenBench-MC provides a single execution pipeline where hardware is automatically detected. It injects hardware-specific prompt templates and applies adaptive numerical tolerances and anti-hack configurations per platform (details in Appendix C), creating the first standardized arena for heterogeneous Triton evaluation.

3.3 Evaluation Framework

Both sub-benchmarks are powered by a shared evaluation infrastructure that executes the following workflow. The distributed sandbox infrastructure is detailed in Appendix D.

Kernel Generation. We evaluate methods across two paradigms: (1) *LLM Sampling Methods*—direct zero-shot and multi-sample generation (Pass@1, Pass@5) [27] without execution feedback (pseudocode in Appendix E), evaluating Opus-4.6, GLM-5.0, Qwen3.5-27B, and MiniMax M-2.5; (2) *Agent-Based Methods*—autonomous coding agents with execution feedback loops, including vanilla agentic frameworks (Claude Code, OpenCode) and kernel-specialized agents (AutoKernel [28], AKO4all [29], CUDA Optimized Skill [30]).

Verification. Generated kernels serve as input to the unified `verify` engine. Unlike prior benchmarks that evaluate fixed single-shape inputs [6], we construct a combinatorial test suite of k_i distinct test cases per operator by taking the Cartesian product of core semantic parameters (e.g., *dim*, *transpose*) alongside shape, data type, and memory layout (details in Appendix F). Kernels are mounted directly into PyTorch’s dispatch tree via `torch.library`, ensuring drop-in replaceability. A kernel is considered correct only when it passes numerical validation across *all* k_i test cases.

³All vLLM evaluations use v0.13.0 in this paper.

⁴All cuBLAS evaluations use v12.4 in this paper.

Anti-Hack Architecture. To guard against **benchmark hacking** [31] (e.g., bypassing Triton compilation by calling pre-compiled backend APIs), we deploy a three-tier interception mechanism:

- *L1: AST Static Scan:* Parses the generated abstract syntax tree to block blacklisted calls (e.g., `torch.ops.aten.*`, `import vllm`, and `ctypes`) and dynamic injection bypasses.
- *L2: Ghost Replay:* First executes the kernel normally to capture outputs, then replaces the `@triton.jit`-decorated function with a no-op in memory and replays; identical outputs logically prove the Triton kernel was never invoked, triggering a cheating flag.
- *L3: Hardware Profiling:* Uses `torch.profiler` to ensure Triton-specific signatures exist in low-level trace logs. This layer operates exclusively on NVIDIA hardware; heterogeneous platforms rely on the first two layers due to the absence of equivalent profiling tools.

Evaluation Metrics. We evaluate correctness, performance, and cost efficiency. Clean pass rate counts an operator as successful only when all k_i test cases pass both numerical validation and the three-layer anti-hack check. For performance, we compute a two-level geometric mean [32, 33]: first across the k_i test cases to obtain operator-level speedup S_i , then across all operators to obtain the global speedup. Speedup distribution metrics (fast_p) are detailed in Appendix G. For agentic methods, we track total tokens, tokens per success (total tokens divided by passed operators), and total wall time (cumulative per-operator solving duration, parallelism-agnostic).

4 Experiments and Evaluation

In this section, we deploy KernelGenBench to evaluate LLMs and agentic frameworks across diverse operator sources and heterogeneous hardware platforms.

4.1 Experimental Setup

We deploy the evaluation suite across six dedicated full-node hardware platforms: NVIDIA A100 and five alternative hardware platforms (Platform A–E). The full 210-operator problem set is evaluated on the NVIDIA baseline, while the 110-operator ATen subset is used to benchmark cross-platform portability. For all language models, we use `temperature=0` for Pass@1 generation and `temperature=0.8` for Pass@5, with a context window limit of `max_tokens=16384` and a unified 30-minute wall-clock timeout per operator task. Evaluated paradigms are detailed in Section 3.3. Results for legacy models are deferred to Appendix H. All agent results are single-trajectory measurements due to cost constraints; we therefore interpret cross-method differences below 5 pp cautiously.

4.2 KernelGenBench-MS: Multi-Source Evaluation

We evaluate all methods on NVIDIA A100 across the full 210-operator suite. Table 2 presents the comprehensive results.

Finding 1: Model and Method Capabilities. Claude Code (Opus-4.6) achieves the highest overall accuracy at 87%, while AutoKernel (Qwen3.5-27b) achieves the highest overall speedup at $1.02\times$. Comparing the two top performers shows a fundamental trade-off: Claude Code outperforms the kernel-specialized AKO4all (Opus-4.6) at 83% accuracy by 4 pp, but AKO4all achieves $0.97\times$ speedup versus Claude Code’s $0.78\times$. Kernel-specialized agents prioritize performance optimization over correctness, sacrificing functional correctness on edge cases to maximize speedup. By contrast, vanilla agentic frameworks allocate more iterations to debugging and correctness verification, producing higher pass rates at the cost of performance. Speedup distribution metrics (fast_p) for all configurations are detailed in Appendix I.

Breaking down by operator source shows where specialization pays off. On ATen operators, both Claude Code and OpenCode with Opus-4.6 reach 92% accuracy, while kernel-specialized agents (AKO4all, AutoKernel with GLM-5.0/Qwen3.5-27b) achieve $1.00\times$ speedup, perfectly matching the baseline. On vLLM operators, Claude Code (Opus-4.6) leads in accuracy at 68%, but AutoKernel (Qwen3.5-27b) delivers $1.63\times$ speedup—the highest across all sources—demonstrating that specialized agents perform well at performance optimization on complex operators. On cuBLAS operators, Claude Code (Opus-4.6) achieves 94% accuracy, but even the best speedup (AutoKernel Qwen at

Table 2: NVIDIA A100 evaluation across 210 operators from three sources (ATen, vLLM, cuBLAS), showing accuracy and speedup by operator source across all generation paradigms.

Method & Setup	Overall (210)		ATen (110)		vLLM (50)		cuBLAS (50)	
	Acc (%)	Spd (×)	Acc (%)	Spd (×)	Acc (%)	Spd (×)	Acc (%)	Spd (×)
LLM Sampling Methods								
Pass@1 (Opus-4.6)	41	0.70	39	0.90	20	0.76	68	0.49
Pass@1 (GLM-5.0)	21	0.68	21	0.49	24	1.24	20	0.73
Pass@1 (Qwen3.5-27b)	7	0.85	8	0.83	2	2.05	8	0.71
Pass@1 (MiniMax M-2.5)	2	0.88	4	0.88	0	0.00	0	0.00
Pass@5 (Opus-4.6)	57	0.68	62	0.79	28	0.71	74	0.49
Pass@5 (GLM-5.0)	36	0.77	45	0.64	32	1.28	20	0.76
Pass@5 (Qwen3.5-27b)	11	<u>1.01</u>	13	1.04	12	0.70	8	0.68
Pass@5 (MiniMax M-2.5)	17	0.69	21	0.76	18	1.27	2	0.46
Vanilla Agentic Frameworks								
Claude Code (Opus-4.6)	87	0.78	92	0.86	68	1.02	94	0.51
Claude Code (GLM-5.0)	67	0.83	72	0.88	52	1.23	72	0.53
Claude Code (Qwen3.5-27b)	62	0.70	80	0.68	38	1.17	48	0.50
Claude Code (MiniMax M-2.5)	49	0.69	69	0.78	26	0.46	26	0.58
OpenCode (Opus-4.6)	81	0.73	92	0.82	46	0.97	92	0.50
OpenCode (GLM-5.0)	72	0.69	87	0.71	42	0.94	70	0.51
OpenCode (Qwen3.5-27b)	53	0.78	58	0.75	44	1.31	52	0.58
OpenCode (MiniMax M-2.5)	41	0.62	50	0.77	26	0.44	36	0.42
Kernel-Specialized Agents								
AKO4all (Opus-4.6)	83	0.97	91	<u>1.00</u>	64	1.62	84	0.61
CUDA Opt. Skill (MiniMax M-2.5)	45	0.80	63	0.81	24	0.92	28	0.45
AutoKernel (GLM-5.0)	71	0.99	87	<u>1.00</u>	43	1.40	66	0.75
AutoKernel (Qwen3.5-27b)	47	1.02	69	<u>1.00</u>	16	<u>1.63</u>	30	0.80
AutoKernel (MiniMax M-2.5)	43	0.89	66	0.87	20	1.52	16	0.46

0.80×) remains below the proprietary baseline. Model-specific generation bottlenecks and anti-hack interception patterns are detailed in Appendix J.

Finding 2: Operator-Source Difficulty Hierarchy. A strict, model-agnostic difficulty hierarchy emerges across the three operator sources. ATen operators are the most tractable: even weaker models achieve reasonable accuracy (e.g., Claude Code with MiniMax reaches 69%), and speedup consistently clusters around 0.8–1.0×, matching the framework baseline. This reflects ATen’s design as a high-level framework API with well-defined semantics and moderate performance requirements.

vLLM operators present the opposite profile—functional correctness is extremely challenging (accuracy drops sharply, e.g., Claude Code with MiniMax falls to 26%), but when successfully generated, they deliver genuine acceleration potential (AKO4all achieves 1.62×, AutoKernel with Qwen reaches 1.63×). This difficulty is due to vLLM’s complex inference-specific operators (paged attention, KV cache management, mixed-precision quantization), which require intricate memory layouts and algorithmic understanding that LLMs struggle to implement correctly. However, the baseline implementations are not heavily hand-tuned, leaving room for optimization when correctness is achieved.

cuBLAS operators occupy the middle ground: moderate accuracy is achievable (Claude Code with Opus reaches 94%), but speedup is universally capped—virtually all configurations cluster tightly around 0.50×, unable to exceed the hand-tuned proprietary baseline. This performance ceiling reflects cuBLAS’s status as a closed-source, heavily optimized library representing decades of expert engineering. The baseline directly loads `libcublas.so` via `ctypes`, bypassing all high-level wrappers, making it nearly impossible for LLM-generated Triton kernels to match proprietary BLAS performance. This hierarchy shows that ATen serves as the tractable baseline, vLLM tests optimization capability on complex kernels, and cuBLAS exposes the fundamental difficulty of matching closed-source performance.

4.3 KernelGenBench-MC: Cross-Platform Evaluation

We extend the evaluation of the 110 ATen operators across six hardware platforms. Table 3 presents the comprehensive results.

Table 3: Cross-platform evaluation on 110 ATen operators across six hardware platforms, showing whether correctness and speedup transfer across heterogeneous hardware backends.

Method & Setup	NVIDIA		Platform A		Platform B		Platform C		Platform D		Platform E	
	Acc (%)	Spd (×)	Acc (%)	Spd (×)	Acc (%)	Spd (×)	Acc (%)	Spd (×)	Acc (%)	Spd (×)	Acc (%)	Spd (×)
LLM Sampling Methods												
Pass@1 (Opus-4.6)	39	0.90	46	0.19	44	0.69	37	0.98	38	0.89	38	0.88
Pass@1 (Qwen3.5-27b)	8	0.83	9	0.09	3	1.02	7	0.90	7	0.98	10	1.03
Pass@1 (MiniMax M-2.5)	4	0.88	4	0.25	6	<u>1.33</u>	4	<u>1.05</u>	5	0.77	4	1.15
Pass@5 (Opus-4.6)	62	0.79	63	0.15	60	0.74	54	0.92	65	0.68	57	0.83
Pass@5 (Qwen3.5-27b)	13	1.04	16	0.18	11	1.10	15	0.72	10	0.99	17	1.02
Pass@5 (MiniMax M-2.5)	21	0.76	17	0.20	15	0.53	12	<u>1.05</u>	8	0.33	9	0.76
Vanilla Agentic Frameworks												
Claude Code (Opus-4.6)	92	0.86	89	0.18	93	0.80	88	0.87	96	0.89	83	0.83
Claude Code (GLM-5.0)	67	0.83	65	0.16	65	0.96	65	0.81	59	0.90	37	0.77
Claude Code (Qwen3.5-27b)	80	0.68	78	0.25	75	0.61	75	0.85	82	0.77	23	0.81
Claude Code (MiniMax M-2.5)	69	0.78	69	0.16	74	0.59	73	0.72	83	0.63	69	0.58
Kernel-Specialized Agents												
AKO4all (Opus-4.6)	89	<u>1.00</u>	84	0.30	88	1.09	88	1.08	86	1.12	80	<u>1.07</u>
CUDA Opt. Skill (MiniMax M-2.5)	63	0.81	53	0.21	64	0.77	65	0.81	67	0.77	58	0.79
AutoKernel (GLM-5.0)	87	<u>1.00</u>	53	0.82	56	1.01	64	0.99	59	1.00	25	1.01
AutoKernel (Qwen3.5-27b)	69	<u>1.00</u>	40	0.37	75	1.03	65	<u>1.00</u>	74	<u>1.04</u>	21	1.01
AutoKernel (MiniMax M-2.5)	66	0.87	61	<u>0.66</u>	71	1.36	66	0.99	71	<u>1.04</u>	50	1.02

Finding 1: Model and Method Capabilities Across Platforms. For accuracy, Claude Code (Opus-4.6) consistently achieves the highest rates across most platforms: 92% on NVIDIA, 89% on Platform A, 93% on Platform B, 88% on Platform C, and 96% on Platform D; Platform E is the exception, where Claude Code (MiniMax M-2.5) reaches 69%. For speedup, kernel-specialized agents dominate: AKO4all (Opus-4.6) achieves the best overall performance with 1.00× on NVIDIA, 1.08× on Platform C, 1.12× on Platform D, and 1.07× on Platform E, while AutoKernel (MiniMax M-2.5) reaches the highest single-platform speedup at 1.36× on Platform B. However, kernel-specialized agents exhibit severe accuracy variance across platforms: AKO4all ranges from 89% on NVIDIA to 80% on Platform E, while AutoKernel (Qwen3.5-27b) spans 75% on Platform B down to 21% on Platform E, showing that even state-of-the-art specialized methods struggle with cross-platform portability.

This divergence is due to how methods utilize platform-specific information. Vanilla agentic frameworks like Claude Code provide minimal initial context but include hardware-specific constraints (e.g., API limitations, type strictness) in the prompt; agents actively leverage this information during debugging, adapting their implementations to platform quirks. Kernel-specialized agents, by contrast, focus heavily on performance tuning (profiling, block-size search, memory-access optimization) and often overlook the provided platform constraints, leading to compilation failures or runtime errors on non-NVIDIA backends despite achieving higher speedup when kernels do compile successfully.

Finding 2: Platform-Specific Performance Divergence. Cross-platform evaluation exposes severe performance heterogeneity. Platform A suffers from a catastrophic speedup collapse: Claude Code (Opus-4.6) maintains 89% accuracy but achieves only 0.18× speedup, the lowest across all platforms. This 4.8× degradation relative to NVIDIA (0.86×) exposes unoptimized backend implementations despite functional correctness. Platform E exhibits the opposite failure mode—accuracy collapse: Claude Code with Qwen3.5-27b drops to 23%, and AutoKernel variants fall to 21–25%, significantly lower than other platforms (typically 60–90%). This reflects immature vendor compilers that frequently hang or crash when processing unstructured LLM-generated code, leading to compilation timeouts. Furthermore, non-NVIDIA platforms incur massive compilation overheads: Platform A requires 2.1× tokens and 2.0× time relative to NVIDIA (Figure 2), forcing agents to burn iteration budgets on compilation debugging rather than kernel optimization.

Finding 3: Cross-Platform Cost Overhead. Non-NVIDIA platforms incur massive compilation overheads, as quantified in Figure 2 (right panel). Platform A exhibits the most severe overhead: 173M total tokens (2.06× NVIDIA’s 84M baseline) and 18 hours (2.00× NVIDIA’s 9 hours). Platform B requires 128M tokens (1.52×) and 16 hours (1.78×), while Platforms C, D, and E consume

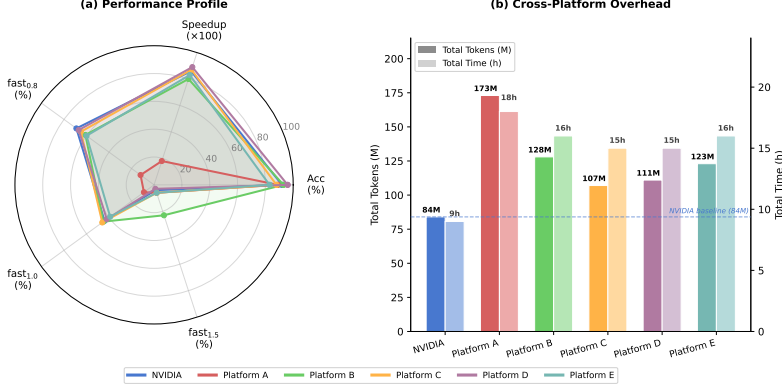


Figure 2: Platform A collapses speedup to 0.18 $\times$ despite high accuracy, while non-NVIDIA platforms incur up to 2 $\times$ compilation overhead. Left: radar chart showing accuracy and speedup across six platforms (110 ATen, Claude Code Opus-4.6). Right: token and time overhead relative to NVIDIA.

107–123M tokens (1.27–1.46 $\times$) and 15–16 hours (1.67–1.78 $\times$). This overhead is not algorithmic; it is purely ecosystem friction from immature vendor compilers and incomplete Triton backend support.

The root cause is that models lack prior exposure to heterogeneous hardware constraints during pretraining. When generating kernels for alternative platforms, agents must iteratively discover platform-specific limitations through trial and error: API availability (e.g., missing `tl.acosh` or `tl.math.tanh`), type system strictness (mixed int32/int64 loops that compile silently on CUDA but raise hard errors elsewhere), pointer addressing modes (32-bit vs 64-bit), and LLVM IR compatibility gaps. Each compilation failure forces the agent to burn tokens diagnosing opaque backend errors, adjusting code to satisfy undocumented constraints, and re-attempting compilation, consuming iteration budgets that would otherwise be spent on functional correctness or performance optimization. Platform A’s 2 $\times$ overhead directly reflects its backend’s fragility: frequent compilation hangs and cryptic error messages force agents into extended debugging cycles, while Platform E’s compiler instability (leading to the 21–25% accuracy collapse noted in Finding 2) similarly inflates token costs as agents repeatedly retry failed compilations before timing out.

4.4 Accuracy-Speedup Gap

Figure 3 plots per-operator accuracy against speedup on the NVIDIA baseline across 16 configurations. A systematic pattern emerges: accuracy spans the full range (2% for MiniMax Pass@1 to 87% for Claude Code Opus-4.6), while speedup clusters tightly in a narrow band (0.62–1.01 $\times$, with 14 of 16 configurations falling within 0.68–0.83 $\times$). The two outliers are Qwen3.5 Pass@5 at 1.01 $\times$ (11% accuracy) and MiniMax OpenCode at 0.62 $\times$ (41% accuracy). This divergence indicates survivorship bias: operators that weaker models fail to solve are disproportionately the computationally complex ones, where the baseline is heavily hand-tuned and high speedup is hardest to achieve. Weaker configurations report higher average speedup not because they optimize better, but because they fail the complex tasks and avoid their severe performance penalties (e.g., the 0.50 $\times$ ceiling of cuBLAS). They only survive the simpler operators where matching the baseline (achieving $\sim$ 0.8–1.0 $\times$) is relatively easy. Consequently, speedup comparisons across methods are only meaningful when conditioned on matched operator subsets.

4.5 Trajectory Analysis

Analysis of hundreds of complete LLM and agent trajectories uncovers two distinct failure layers. **Universal algorithmic failures** occur across all platforms: infinite dispatch recursion (calling the overridden ATen operator internally triggers unbounded recursion), hallucinated Triton APIs (models generate calls to non-existent functions such as `tl.pow`, `tl.einsum`, `tl.gather`), and algorithmically hard operators (`matmul`, `sort`, `cumsum` require cross-block parallel algorithms that agents rarely converge to).

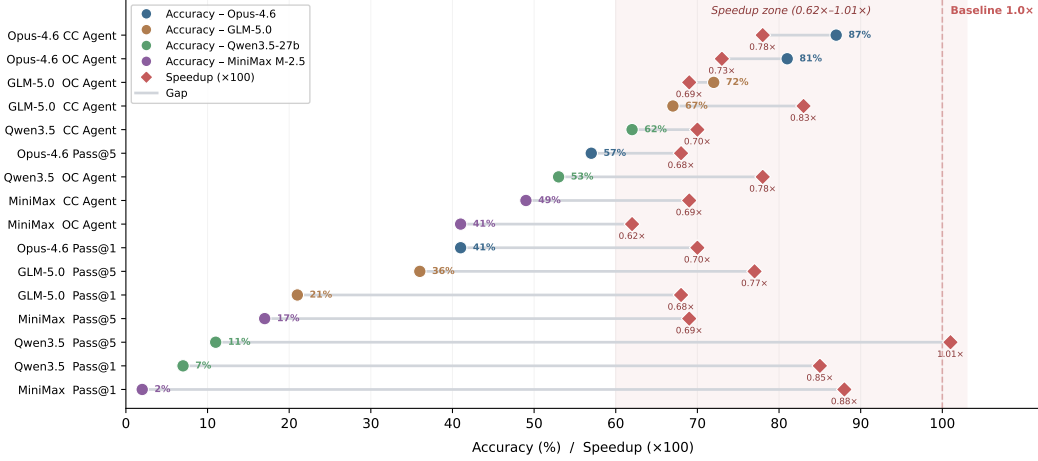


Figure 3: Accuracy–speedup divergence: accuracy spans the full range while speedup clusters in a narrow band—“accuracy leaps, speedup stalls.”

Heterogeneous-platform failures directly explain the accuracy collapse on Platform E: LLVM IR incompatibility triggers `PassManager::run` failed errors (31 observed occurrences on one platform), 32-bit pointer addressing causes memory errors on large-tensor operators, and missing math APIs (`tl.acosh`, `tl.math.tanh`) must be manually reimplemented. Further failure patterns (8 universal and 6 platform-specific) are provided in Appendix K.

4.6 Agentic Cost Efficiency

The introduction of closed-loop execution brings significant economic and time overhead. Table 4 quantifies this cost on the NVIDIA A100 baseline. Kernel-specialized agents universally consume far more tokens and time than vanilla frameworks—their extended iteration budgets are spent optimizing kernel performance. AKO4all is the most extreme case, requiring 904M tokens and 83 hours—over $3\times$ the token cost of Claude Code (Opus-4.6) at 263M—achieving 5.19M tokens per successful operator. Across all kernel-specialized methods (AKO4all, CUDA Optimized Skill, AutoKernel variants), the average tokens per successful operator is 5.11M, orders of magnitude higher than vanilla agentic frameworks (1.45–3.30M) and simple LLM sampling approaches. These results highlight the need for more cost-efficient agentic methods that can close the performance gap without prohibitive overhead.

Table 4: Agentic cost on NVIDIA A100 (210 operators). Total Tokens (M), Tokens per Successful Operator (M), and Total Time (h).

Method	Total Tokens (M)	Tokens per Success (M)	Total Time (h)
Claude Code (Opus-4.6)	263	1.45	33
Claude Code (GLM-5.0)	243	1.67	45
Claude Code (Qwen3.5-27b)	381	2.93	48
Claude Code (MiniMax M-2.5)	340	3.30	50
AKO4all (Opus-4.6)	904	5.19	83
CUDA Opt. Skill (MiniMax M-2.5)	594	6.75	97
AutoKernel (GLM-5.0)	471	3.16	102
AutoKernel (Qwen3.5-27b)	475	4.80	102
AutoKernel (MiniMax M-2.5)	508	5.64	105

4.7 Ablation: Value of Execution Feedback

To isolate the contribution of execution feedback, we conduct a controlled ablation on Opus-4.6 across the full 210-operator NVIDIA suite. Figure 4 presents the results; the full per-subset breakdown is in Appendix L.

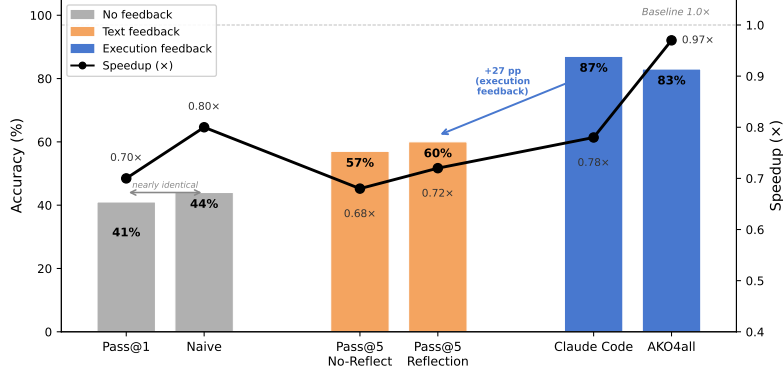


Figure 4: Ablation study on Opus-4.6 (NVIDIA A100, 210 operators). Execution feedback (+27.2 pp from Pass@5 Reflection to Claude Code) is the key driver of agentic value.

Performance scales tightly with method complexity: Pass@1 (41%) < Pass@5 (57%) < Vanilla Agentic (87%) $\approx$ Kernel-Specialized (83%). Merely placing the model in a naive framework wrapper adds no value: Single-step Agent achieves 44%, nearly identical to Pass@1 at 41%. Autonomous interactive debugging is the true driver of agentic value: Claude Code reaches 87%, a 27 pp jump over standard reflection (Pass@5 Reflection at 60%).

We further identify an *exploitation–exploration* trap in fixed traceback-based reflection. Even when provided with execution errors, Pass@5 Reflection barely outperforms independent multi-sampling (No-Reflect at 57%) and actually underperforms it on vLLM (24% vs 28%). Code similarity analysis shows that traceback-driven reflection produces 88% average cross-turn similarity (local patching), whereas independent sampling averages 36% (global search). Without the agentic autonomy to write custom test harnesses or dynamically probe intermediate variables, models fed with raw tracebacks over-exploit their first-round implementation and easily get trapped in local optima.

5 Conclusion

We have presented **KernelGenBench**, a unified benchmark comprising two complementary sub-benchmarks: KernelGenBench-MS (210 operators from three sources) and KernelGenBench-MC (cross-platform verification) for systematically evaluating LLM-driven Triton kernel generation. Our large-scale evaluation shows three findings: agent-based methods consistently outperform pure LLM sampling approaches, although cuBLAS operators remain the most challenging across all methods; generation accuracy varies dramatically across hardware platforms, with even state-of-the-art kernel-specialized agents experiencing severe cross-platform degradation (e.g., AutoKernel drops from 87% on NVIDIA to 25% on Platform E); and specialized agentic methods incur token costs averaging 5.11 million per successful operator (AKO4all reaches 5.19 million), orders of magnitude higher than simple LLM sampling approaches.

Overall, KernelGenBench provides the first comprehensive framework for understanding and advancing LLM-driven kernel generation across diverse operator sources and heterogeneous hardware platforms. We open-source the full benchmark suite, evaluation infrastructure, and all agent trajectories to facilitate reproducibility and community extension. Three limitations bound current conclusions. First, the full Multi-Chip $\times$ Multi-Source benchmark is absent due to nascent vendor ecosystems, and the current 210-operator suite covers only a fraction of the full ATen and library surface. Second, single-trajectory evaluation is cost-constrained: with over 15 billion tokens consumed, repeated trials are economically prohibitive. Third, the benchmark measures the joint performance of an LLM and its agentic scaffold; systematically decoupling framework heuristics from intrinsic LLM capabilities remains an open challenge. Future work will partner with hardware vendors to complete this unified benchmark, scale toward 900+ ATen operators and additional libraries (cuDNN [34], CUTLASS [35], SGLang [36]), and introduce specialized subsets to assess fine-grained peak performance.

Acknowledgments and Disclosure of Funding

The authors would like to thank BAAI for providing the computational resources and API tokens that supported this research. We are also grateful to Yang Yu and Liangdong Wang for their insightful discussions and valuable assistance throughout the development of this work. Special thanks go to Yipeng Jiang for her contribution to the illustrations and the overview figure in this paper. Additionally, we express our sincere gratitude to the participating hardware vendors for their continuous communication, technical feedback, and collaborative support in co-building this benchmarking ecosystem.

References

- [1] Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. Dr. kernel: Reinforcement learning done right for triton kernel generations. *CoRR*, abs/2602.05885, 2026. doi: 10.48550/ARXIV.2602.05885. URL <https://doi.org/10.48550/arXiv.2602.05885>.
- [2] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *CoRR*, abs/2108.07732, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [3] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- [4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.
- [5] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/7ede97c3e082c6df10a8d6103a2eebd2-Abstract-Conference.html.
- [6] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient GPU kernels? In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- [7] PyTorch Team. Tritonbench: Evaluating triton kernel generation capabilities. <https://github.com/meta-pytorch/tritonbench>, 2025. GitHub repository.
- [8] Vasily Volkov. Better performance at lower occupancy. In *Proceedings of the GPU technology conference, GTC*, volume 10, page 16. San Jose, CA, 2010. URL https://www.nvidia.com/content/gtc-2010/pdfs/2238_gtc2010.pdf.
- [9] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. Triton: an intermediate language and compiler for tiled neural network computations. In Tim Mattson, Abdullah Muzahid, and Armando Solar-Lezama, editors, *Proceedings of the 3rd ACM SIGPLAN International*

Workshop on Machine Learning and Programming Languages, MAPL@PLDI 2019, Phoenix, AZ, USA, June 22, 2019, pages 10–19. ACM, 2019. doi: 10.1145/3315508.3329973. URL <https://doi.org/10.1145/3315508.3329973>.

- [10] Jason Ansel, Edward Z. Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafir, editors, *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, pages 929–947. ACM, 2024. doi: 10.1145/3620665.3640366. URL <https://doi.org/10.1145/3620665.3640366>.
- [11] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 8024–8035, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- [12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace, editors, *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- [13] Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, Guang Liu, and Yonghua Lin. Towards automated kernel generation in the era of llms, 2026. URL <https://arxiv.org/abs/2601.15727>.
- [14] Zacharias V. Fisches, Sahan Paliskara, Simon Guo, Alex Zhang, Joe Spisak, Chris Cummins, Hugh Leather, Gabriel Synnaeve, Joe Isaacson, Aram Markosyan, and Mark Saroufim. Kernel-llm: Making kernel development more accessible, 6 2025. URL <https://huggingface.co/facebook/KernelLLM>.
- [15] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-turn RL for generating CUDA kernels. *CoRR*, abs/2507.11948, 2025. doi: 10.48550/ARXIV.2507.11948. URL <https://doi.org/10.48550/arXiv.2507.11948>.
- [16] Jian Yang, Wei Zhang, Jiajun Wu, Junhang Cheng, Shawn Guo, Haowen Wang, Weicheng Gu, Yaxin Du, Joseph Li, Fanglin Xu, Yizhi Li, Lin Jing, Yuanbo Wang, Yuhao Gao, Ruihao Gong, Chuan Hao, Ran Tao, Aishan Liu, Tuney Zheng, Ganqu Cui, Zhoujun Li, Mingjie Tang, Chenghua Lin, Wayne Xin Zhao, Xianglong Liu, Ming Zhou, Bryan Dai, and Weifeng Lv. Incoder-32b: Code foundation model for industrial scenarios. *CoRR*, abs/2603.16790, 2026. doi: 10.48550/ARXIV.2603.16790. URL <https://doi.org/10.48550/arXiv.2603.16790>.
- [17] Weinan Dai, Hanlin Wu, Qiying Yu, Huan-Ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. CUDA agent: Large-scale agentic RL for high-performance

- CUDA kernel generation. *CoRR*, abs/2602.24286, 2026. doi: 10.48550/ARXIV.2602.24286. URL <https://doi.org/10.48550/arXiv.2602.24286>.
- [18] Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, Yu-Jung Chen, Hanfeng Chen, Aditya Kane, Ronny Krashinsky, Ming-Yu Liu, Vinod Grover, Luis Ceze, Roger A. Bringmann, John Tran, Wei Liu, Fung Xie, Michael Lightstone, and Humphrey Shi. AVO: agentic variation operators for autonomous evolutionary search. *CoRR*, abs/2603.24517, 2026. doi: 10.48550/ARXIV.2603.24517. URL <https://doi.org/10.48550/arXiv.2603.24517>.
 - [19] Alec M. Hammond, Aram Markosyan, Aman Dontula, Simon Mahns, Zacharias V. Fisches, Dmitrii Pedchenko, Keyur Muzumdar, Natacha Supper, Mark Saroufim, Joe Isaacson, Laura Wang, Warren Hunt, Kaustubh Gondkar, Roman Levenstein, Gabriel Synnaeve, Richard Li, Jacob Kahn, and Ajit Mathews. Agentic operator generation for ML asics. *CoRR*, abs/2512.10977, 2025. doi: 10.48550/ARXIV.2512.10977. URL <https://doi.org/10.48550/arXiv.2512.10977>.
 - [20] PyTorch Team. KernelAgent: Hardware-Guided GPU Kernel Optimization via Multi-Agent Orchestration. <https://pytorch.org/blog/kernelagent-hardware-guided-gpu-kernel-optimization-via-multi-agent-orchestration/>, 2025. PyTorch Blog. Accessed: 2026-05-05.
 - [21] Mark Saroufim, Jiannan Wang, Bert Maher, Sahan Paliskara, Laura Wang, Shahin Sefati, and Manuel Candales. Backendbench: An evaluation suite for testing how well llms and humans can write pytorch backends, 2025. URL <https://github.com/meta-pytorch/BackendBench>.
 - [22] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
 - [23] Shanli Xing, Yiyan Zhai, Alexander Jiang, Yixin Dong, Yong Wu, Zihao Ye, Charlie Ruan, Yingyi Huang, Yineng Zhang, Liangsheng Yin, Aksara Bayyapu, Luis Ceze, and Tianqi Chen. Flashinfer-bench: Building the virtuous cycle for ai-driven LLM systems. *CoRR*, abs/2601.00227, 2026. doi: 10.48550/ARXIV.2601.00227. URL <https://doi.org/10.48550/arXiv.2601.00227>.
 - [24] Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qijing Huang, Zhifan Ye, Nestor Qin, Fengzhe Zhou, Yuan Zhang, Jingquan Wang, Sana Damani, Dheeraj Peri, Ouye Xie, Aditya Kane, Moshe Maor, Michael Behar, Triston Cao, Rishabh Mehta, Vartika Singh, Vikram Sharma Mailthody, Terry Chen, Zihao Ye, Hanfeng Chen, Tianqi Chen, Vinod Grover, Wei Chen, Wei Liu, Eric Chung, Luis Ceze, Roger Bringmann, Cyril Zeller, Michael Lightstone, Christos Kozyrakis, and Humphrey Shi. Sol-execbench: Speed-of-light benchmarking for real-world gpu kernels against hardware limits, 2026. URL <https://arxiv.org/abs/2603.19173>.
 - [25] Zhongzhen Wen, Yinghui Zhang, Zhong Li, Zhongxin Liu, Linna Xie, and Tian Zhang. Multikernelbench: A multi-platform benchmark for kernel generation. *CoRR*, abs/2507.17773, 2025. doi: 10.48550/ARXIV.2507.17773. URL <https://doi.org/10.48550/arXiv.2507.17773>.
 - [26] FlagOS contributors. Flagos/flaggems: An operator library for large language models implemented in the triton language., 2024. URL <https://github.com/flagos-ai/FlagGems>.
 - [27] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgan Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie

- Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [28] Jaber Jaber and Osama Jaber. Autokernel: Autonomous GPU kernel optimization via iterative agent-driven search. *CoRR*, abs/2603.21331, 2026. doi: 10.48550/ARXIV.2603.21331. URL <https://doi.org/10.48550/arXiv.2603.21331>.
- [29] TongmingLAIC. AKO4ALL. <https://github.com/TongmingLAIC/AKO4ALL>, 2026.
- [30] KernelFlow-ops. cuda-optimized-skill. <https://github.com/KernelFlow-ops/cuda-optimized-skill>, 2026.
- [31] Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. Tritonrl: Training llms to think and code triton without cheating. *CoRR*, abs/2510.17891, 2025. doi: 10.48550/ARXIV.2510.17891. URL <https://doi.org/10.48550/arXiv.2510.17891>.
- [32] Standard Performance Evaluation Corporation (SPEC). SPEC CPU 2017 Benchmark Suite. <https://www.spec.org/cpu2017/>, 2017.
- [33] Philip J. Fleming and John J. Wallace. How not to lie with statistics: The correct way to summarize benchmark results. *Commun. ACM*, 29(3):218–221, 1986. doi: 10.1145/5666.5673. URL <https://doi.org/10.1145/5666.5673>.
- [34] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cudnn: Efficient primitives for deep learning. *CoRR*, abs/1410.0759, 2014. URL <http://arxiv.org/abs/1410.0759>.
- [35] Andrew Kerr, Duane Merrill, Julien Demouth, and John Tran. Cutlass: Fast linear algebra in cuda c++, 2017. URL <https://github.com/NVIDIA/cutlass>.
- [36] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.
- [37] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 29304–29320, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html>.
- [38] Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H. Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, Afroz Mohiuddin, Ryan Sepassi, George Tucker, and Henryk Michalewski. Model based reinforcement learning for atari. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=S1xCPJHtDB>.

A Prompt Design

KernelGenBench dynamically constructs structured prompts for each operator rather than relying on vague natural-language descriptions. Table 5 summarizes the prompt template modules and their differences between the LLM Sampling and Agentic evaluation paradigms.

Table 5: Prompt template modules: LLM Sampling vs. Agentic Frameworks.

Module	LLM Sampling	Agentic
System Role	✓	✓
One-shot Example	✓	✓
Operator Spec / Baseline Source Injection	✓	✓
Function Signature & Type Constraints	✓	✓
Anti-Cheat Rules	✓	✓
Testing Environment Description	✓	✓
Device-Specific Constraints (Heterogeneous)	✓	✓
Accelerator Device Assignment (Device ID)	—	✓
Verification CLI Command	—	✓
Iterative Workflow Guide	—	✓
Performance Optimization Guide	—	✓

The key difference across the three subsets lies in the “Operator Spec / Baseline Source Injection” module: the ATen subset extracts the full `FunctionSchema` and all overload variants at runtime; the vLLM subset, lacking public API documentation, directly injects the complete Python source code containing `_custom_ops` calls; the cuBLAS subset injects a `ctypes`-based C API call chain (including handle management and pointer mode switching logic).

Figure 5 uses `aten::add` as an example, showing side-by-side the actual prompts under the LLM Sampling and Agentic paradigms (shared portions abbreviated as `[...]` for brevity). The prompts shown are for NVIDIA; on alternative platforms, the device-specific references (e.g., “GPU programmer”) are automatically replaced with the corresponding platform-specific terminology, and additional device constraints are appended.

B Full Operator List

Tables 6, 7, and 8 list the complete operator names for the three KernelGenBench subsets.

C Cross-Platform Prompt and Tolerance Configuration

This section details the hardware-specific configurations injected by KernelGenBench-MC for heterogeneous chip evaluation.

Hardware-Aware Prompt Templates. The framework maintains separate prompt template directories per device type (e.g., `templates/platform_a/`, `templates/platform_b/`), automatically selecting and assembling the corresponding template at runtime based on detected hardware (e.g., injecting platform-specific import statements and runtime initialization code).

D Distributed Sandbox Infrastructure

This section details the production-grade dispatch registration and the process isolation and distributed scheduling infrastructure underlying KernelGenBench’s full-node parallel evaluation.

Production-Grade Dispatch Registration. For ATen operators, the system uses `register_scanner.py` to perform AST scanning and capture `@register` decorators, force-mounting Triton kernels into PyTorch’s `torch.library` dispatch tree. All ATen operators are therefore tested under PyTorch’s full low-level dispatch mechanism, not isolated Python function calls.

LLM Sampling Prompt

You are a skilled GPU programmer proficient in Triton. Your task is to generate a Triton kernel function.

Here is an example of a PyTorch function and its corresponding Triton kernel implementation:
[one-shot add example, with kernel + wrapper]

You must strictly adhere to the following specifications:
The Triton kernel should implement:
`add = torch.ops.aten.add`

The ATen operators involved are:

- `add.Tensor`
- `add.Scalar`
- `add.out`
- *[... 16 overload variants in total]*

The Python wrapper functions should have signatures like:

- `def add.Tensor(...)`
- `def add.Scalar(...)`
- *[...]*

Input and Output Args:
`aten::add.Tensor(Tensor self, Tensor other, *, Scalar alpha=1) -> Tensor`
[... full signature for each variant]

CRITICAL REQUIREMENTS:

1. Handle broadcasting correctly
2. Handle non-contiguous tensors correctly

Generate valid Triton code directly without any explanations. Use `'''python'''` format.

Agentic Framework Prompt

Triton Kernel Implementation Task
Operator: `add` Full name: `aten::add`
GPU ID: 0

Runtime Environment
`CUDA_VISIBLE_DEVICES=0`

Operator Specification
[same schema and signature as LLM prompt]

Verification Tool
`python verify_single.py --code kernel.py`
`--op aten::add --output-json`

Iterative Workflow

1. Write implementation to `kernel.py`
2. Run verification command
3. If failed, read error and fix
4. Repeat until all tests pass

Performance Optimization
After correctness is achieved:

- Tune `BLOCK_SIZE`, `num_warps`, `num_stages`
- Re-run verification to confirm speedup

Anti-Cheat Rules
[same rules as LLM prompt]

Figure 5: Side-by-side comparison of LLM Sampling and Agentic prompts for `aten::add`. Shared content is abbreviated with `[...]`.

Table 6: Full operator list for the ATen subset (110 operators).

<code>_index_put_impl_</code>	<code>_local_scalar_dense</code>	<code>_softmax</code>	<code>binary_cross_entropy_with_logits</code>
<code>_to_copy</code>	<code>acosh</code>	<code>add</code>	<code>affine_grid_generator</code>
<code>add_</code>	<code>amin</code>	<code>arange</code>	<code>upsample_nearest2d_backward</code>
<code>argmax</code>	<code>as_strided</code>	<code>asin</code>	<code>log_sigmoid_backward</code>
<code>bernoulli</code>	<code>bitwise_not</code>	<code>bmm</code>	<code>reflection_pad1d_backward</code>
<code>cat</code>	<code>clone</code>	<code>contiguous</code>	<code>rrelu_with_noise_backward</code>
<code>copy_</code>	<code>cos</code>	<code>cosh</code>	<code>unsafe_split_with_sizes</code>
<code>cumsum</code>	<code>diff</code>	<code>div</code>	<code>smooth_l1_loss_backward</code>
<code>div_</code>	<code>embedding</code>	<code>empty_strided</code>	<code>margin_ranking_loss</code>
<code>eq</code>	<code>erfc</code>	<code>expand</code>	<code>rrelu_with_noise</code>
<code>expand_as</code>	<code>exponential_</code>	<code>fill_</code>	<code>pairwise_distance</code>
<code>floor</code>	<code>floor_divide</code>	<code>fmax</code>	<code>softplus_backward</code>
<code>full</code>	<code>gather</code>	<code>gt</code>	<code>new_empty_strided</code>
<code>hardsigmoid</code>	<code>heaviside</code>	<code>huber_loss</code>	<code>soft_margin_loss</code>
<code>i0</code>	<code>im2col</code>	<code>index</code>	<code>select_backward</code>
<code>index_put_</code>	<code>index_select</code>	<code>item</code>	<code>mish_backward</code>
<code>le</code>	<code>linear</code>	<code>log10</code>	<code>masked_fill_</code>
<code>logaddexp2</code>	<code>logit</code>	<code>matmul</code>	<code>resolve_conj</code>
<code>mean</code>	<code>mish</code>	<code>mm</code>	<code>resolve_neg</code>
<code>mul</code>	<code>narrow</code>	<code>neg</code>	<code>special ENTR</code>
<code>new_ones</code>	<code>ones_like</code>	<code>poisson</code>	<code>scalar_tensor</code>
<code>polygamma</code>	<code>pow</code>	<code>prelu</code>	<code>unsafe_split</code>
<code>renorm</code>	<code>reshape</code>	<code>rot90</code>	<code>unsqueeze</code>
<code>rsqrt</code>	<code>rsub</code>	<code>scatter</code>	<code>zeros_like</code>
<code>select</code>	<code>sgn</code>	<code>silu</code>	<code>softmax</code>
<code>sin</code>	<code>sort</code>	<code>square</code>	<code>stack</code>
<code>sub</code>	<code>sum</code>	<code>t</code>	<code>to</code>
<code>zero_</code>	<code>zeros</code>		

Table 7: Full operator list for the vLLM subset (50 operators, from vLLM v0.13.0).

allspark_w8a16_gemm	apply_repetition_penalties_cuda	awq_gemm
awq_marlin_moe_repack	batched_moe_align_block_size	concat_and_cache_mla
convert_fp8	convert_vertical_slash_indexes	copy_blocks
copy_blocks_mla	cp_gather_cache	cp_gather_indexer_k_quant_cache
cutlass_pack_scale_fp8	cutlass_scaled_mm	cutlass_scaled_mm_azp
fused_add_rms_norm	fused_qk_norm_rope	gather_and_maybe_dequant_cache
ggml_dequantize	ggml_moe_a8	ggml_moe_a8_vec
ggml_mul_mat_a8	ggml_mul_mat_vec_a8	gptq_gemm
gptq_marlin_24_gemm	gptq_marlin_gemm	gptq_marlin_moe_repack
gptq_shuffle	grouped_topk	hadacore_transform
marlin_int4_fp8_preprocess	merge_attn_states	moe_align_block_size
moe_lora_align_block_size	moe_sum	paged_attention_v1
paged_attention_v2	permute_cols	reshape_and_cache
reshape_and_cache_flash	rms_norm	rms_norm_dynamic_per_token_quant
rms_norm_per_block_quant	rotary_embedding	scaled_fp8_quant
scaled_int8_quant	selective_scan_fwd	shuffle_rows
swap_blocks	topk_softmax	

Table 8: Full operator list for the cuBLAS subset (50 operators, based on cuBLAS 12.4).

cublasCcopy_v2	cublasCdotu_v2	cublasCgemmStridedBatched
cublasCgemmStridedBatched_64	cublasCgemm_v2	cublasCgemvBatched_64
cublasCgemvStridedBatched	cublasCgemv_v2	cublasCgeru_v2
cublasCsymm_v2	cublasCsymv_v2	cublasCsyrkEx
cublasDasum_v2	cublasDaxpy_v2	cublasDcopy_v2
cublasDgemmBatched	cublasDgemmStridedBatched	cublasDgemmStridedBatched_64
cublasDgemvBatched	cublasDgemvStridedBatched	cublasDgemv_v2
cublasDsrbmv_v2	cublasDsyr2_v2	cublasDtrsmBatched
cublasHgemmBatched	cublasHgemmStridedBatched	cublasSaxpy_v2
cublasSdgemm	cublasSdot_v2	cublasSgeam
cublasSgemmBatched_64	cublasSgemmEx	cublasSgemmStridedBatched
cublasSgemm_v2	cublasSgemvBatched	cublasSgemvStridedBatched
cublasSger_v2	cublasSscal_v2	cublasSsyrk_v2
cublasStbmv_v2	cublasStrsm_v2	cublasStrsv_v2
cublasZdotc_v2	cublasZgemmBatched	cublasZgemmStridedBatched
cublasZgemvBatched	cublasZgemvStridedBatched	cublasZgerc_v2
cublasZswap_v2	cublasZtrsmBatched	

Full-Node Parallel Evaluation with Process Isolation. To enable full-node utilization, KernelGen-Bench dispatches all operators in parallel across all available accelerator devices on the machine. Each operator runs in a fully isolated subprocess (multiprocessing.spawn), with its own workspace directory and exclusive device assignment—ensuring that a crash, timeout, or device memory corruption in one operator cannot affect any other. The distributed resource scheduler DeviceManager acquires hardware control via kernel-level file locks, recording the holder’s PID and automatically reclaiming locks from crashed processes to prevent permanent device resource leaks. When a subprocess triggers a timeout (300s for correctness, 600s for performance), the parent process sends SIGKILL to destroy the entire process tree.

Adaptive Numerical Tolerance. Numerical comparison of low-level operators is sensitive to floating-point truncation errors. The system implements a dtype- and reduction-dimension-aware adaptive tolerance mechanism: absolute tolerance is computed as $\text{atol} = 10^{-4} \times D_{\text{reduce}}$ (D_{reduce} is the reduction dimension size), and relative tolerance is calibrated by dtype effective-bit resolution (FP32 $\rightarrow 10^{-5}$, FP16 $\rightarrow 10^{-3}$, BF16 $\rightarrow 0.016$). On NVIDIA GPUs, all three anti-hack layers (L1–L3) are enabled; on heterogeneous platforms, due to torch.profiler adaptation limitations, only L1 static scanning and L2 dual-execution comparison are active.

E Evaluation Pipeline Algorithm Details

This section provides the full pseudocode for both the agent dispatch pipeline and the LLM Sampling Methods generation pipeline described in Section 3.3.

Algorithm 1 Vanilla Agentic Framework Evaluation Dispatch

Require: Operator set $\mathcal{O}$, agent backend M , device pool $\mathcal{G}$, timeout $T_{\max}$

```

1: for each  $op \in \mathcal{O}$  in parallel (bounded by  $|\mathcal{G}|$ ) do
2:    $dev \leftarrow \text{DeviceManager.acquire}(\mathcal{G})$  {Works identically on all platforms}
3:    $ws \leftarrow \text{CreateWorkspace}(op)$ 
4:    $prompt \leftarrow \text{BuildPrompt}(op, dev, \text{verify\_cmd})$ 
5:    $proc \leftarrow M.\text{launch}(prompt, ws, dev)$  {Spawn agent process}
6:   wait  $proc$  until completion or  $T_{\max}$ 
7:   {— Agent-internal loop (autonomous) —}
8:   {1. Write kernel.py}
9:   {2. Run verify_single.py  $\rightarrow$  {passed, error}}
10:  {3. If failed: read error, fix code, goto 2}
11:  {4. If passed: optimize performance, re-verify}
12:   $code \leftarrow M.\text{finish}(proc, ws)$  {Extract final kernel}
13:   $\text{DeviceManager.release}(dev)$ 
14: end for
15: return AntiHackCheck(all submitted kernels)

```

Formal definition of the LLM Sampling Methods pipeline: For a given operator, the system samples up to k times sequentially; if the code c_i generated at round i ($i \leq k$) passes correctness verification, the system locks that operator and removes it from the remaining task pool. The Pass@ k score for a single operator is thus the indicator $\mathbb{I}(\exists i \leq k, \text{verify}(c_i) = \text{True})$, and the overall clean pass rate is the mean of these indicators across all operators. In multi-round iteration mode, if round t fails verification, the system extracts the traceback log e_t and constructs the next-round prompt as $p_t = p_{t-1} \oplus c_t \oplus e_t$. The framework maintains separate prompt templates for three scenarios—initial generation, performance optimization, and error repair—to avoid information redundancy and semantic conflicts across stages.

Algorithm 2 LLM Sampling Methods: Pass@K Sequential Early-Stopping Evaluation Pipeline

Require: Operator set $\mathcal{O}$, max sampling rounds K , model policy π

```

1:  $\mathcal{O}_{\text{remaining}} \leftarrow \mathcal{O}, \mathcal{O}_{\text{passed}} \leftarrow \emptyset$ 
2: for  $round = 1$  to  $K$  do
3:   for each  $op \in \mathcal{O}_{\text{remaining}}$  do
4:      $p \leftarrow \text{ConstructPrompt}(op)$  {Dynamically inject schema or source code}
5:      $c \leftarrow \pi(p)$  {Generate code}
6:      $is\_correct, speedup \leftarrow \mathcal{E}.\text{verify}(c)$ 
7:     if  $is\_correct$  then
8:        $\mathcal{O}_{\text{passed}} \leftarrow \mathcal{O}_{\text{passed}} \cup \{op\}$ 
9:     end if
10:  end for
11:   $\mathcal{O}_{\text{remaining}} \leftarrow \mathcal{O}_{\text{remaining}} \setminus \mathcal{O}_{\text{passed}}$  {Active pruning}
12:  if  $\mathcal{O}_{\text{remaining}} == \emptyset$  then
13:    break
14:  end if
15: end for
16: return AntiHackCheck( $\mathcal{O}_{\text{passed}}$ )

```

F Stress Test Shape and Dispatch Configurations

This section details the combinatorial test case construction and shape/stride configurations used in KernelGenBench’s parametrized test cases. All constants are defined in `accuracy_utils.py`.

Extreme Tensor Distributions and Boundary Stress Testing. Test cases deliberately inject many non-aligned long-tail sizes (e.g., 40999, odd boundaries constructed as $2^d + 17$), mixed with full-dimensional coverage from scalar (,) to 5D tensors (16, 7, 57, 32, 29). Minimal-size tests (e.g., shape=(1, 2)) expose kernel launch overhead degradation.

Example: softmax Test Case Construction. For softmax, the semantic parameter is dim (reduction axis). The $k_i = 18$ test cases are the Cartesian product of three axes: shape $\in \{(1, 256), (4096, 256), (200, 2560, 3)\}$, dtype $\in \{\text{float16}, \text{float32}, \text{bfloat16}\}$, and dim $\in \{0, 1\}$. A kernel is counted as passing only if all 18 cases satisfy the adaptive numerical tolerance. The shape and stride grids used across all operator categories are listed in Tables 9 and 10.

Table 9: Shape configurations by operator category.

Category	Shapes
Pointwise	(,), (1,), (1024, 1024), (20, 320, 15), (16, 128, 64, 60), (16, 7, 57, 32, 29)
Reduction	(1, 2), (4096, 256), (200, 40999, 3)
Reduction (small)	(1, 2), (4096, 256), (200, 2560, 3)
BLAS (M, N)	(1, 32), (160, 1024), (5333, 497)
BLAS (M, N, K)	(1, 1, 32), (15, 160, 1024), (495, 5333, 71)
1D (power-of-2)	(16,), (64,), (256,), (1024,)
1D (non-aligned)	(33,), (81,), (273,), (1041,)
Upsample	(32, 16, 128, 128), (15, 37, 256, 256), (3, 5, 127, 127), (128, 192, 42, 51), (3, 7, 1023, 1025)

Table 10: Stride configurations for memory layout stress testing.

Layout Type	Shape	Stride
<i>1D Contiguous</i>		
	(1,)	(1,)
	(1024,)	(1,)
	(1000000,)	(1,)
<i>1D Dilated</i>		
	(1,)	(2,)
	(1024,)	(2,)
	(1000000,)	(2,)
<i>2D Contiguous</i>		
	(1, 1024)	(1024, 1)
	(10000, 128)	(128, 1)
<i>2D Transposed</i>		
	(1024, 1)	(1, 1024)
	(128, 10000)	(1, 128)
<i>3D Contiguous</i>		
	(20, 320, 15)	(4800, 15, 1)
	(200, 40999, 3)	(122997, 3, 1)
<i>3D Transposed</i>		
	(320, 20, 15)	(15, 4800, 1)
	(3, 40999, 200)	(1, 3, 122997)
<i>5D Irregular</i>		
	(10, 10, 10, 10, 10)	(1, 10000, 23, 399, 1024)

G Speedup Statistical Metrics

The analysis pipeline provides three built-in statistical metrics. Given M operators that pass correctness verification in an experiment, where the i -th operator has k_i test cases, the operator-level speedup is defined as:

$$S_i = \left(\prod_{j=1}^{k_i} \frac{T_{\text{base}}^{(i,j)}}{T_{\text{triton}}^{(i,j)}} \right)^{1/k_i} \quad (1)$$

The overall speedup is aggregated via a two-level geometric mean:

$$S_{\text{overall}} = \left(\prod_{i=1}^M S_i \right)^{1/M} \quad (2)$$

In addition to GeoMean, the framework includes two robust auxiliary metrics:

1. Median

Sort $S_1, \dots, S_M$ and take the middle value:

$$\text{Median} = \begin{cases} S_{(M+1)/2} & M \text{ is odd} \\ \frac{1}{2} (S_{M/2} + S_{M/2+1}) & M \text{ is even} \end{cases}$$

The median is immune to extreme outliers and provides a robust central tendency estimate when inter-operator speedup variance is large.

2. Interquartile Mean (IQM)

Discard the lowest 25% and highest 25% of sorted operators, then take the arithmetic mean of the middle 50%:

$$\text{IQM} = \frac{1}{|Q_3 - Q_1|} \sum_{i=Q_1}^{Q_3} S_{(i)}$$

where $Q_1 = \lfloor M/4 \rfloor$, $Q_3 = \lfloor 3M/4 \rfloor$. IQM balances the statistical efficiency of the mean with the robustness of the median, and has been widely adopted in reinforcement learning benchmarks such as Atari 100k [37, 38], particularly for scenarios with limited samples and skewed distributions.

The three metrics serve different purposes: GeoMean is optimal for multiplicative noise and consistent with system evaluation standards such as SPEC CPU; Median provides the most conservative central estimate; IQM yields a more stable mean after trimming tail noise. All main tables in this paper report GeoMean.

H Legacy Model Baselines on NVIDIA A100

Tables 11 and 12 report the static generation performance of legacy and reference foundation models (Opus-4.5, GPT-5.4, GPT-5.2, GLM-4.7) on NVIDIA A100, provided for historical reference and cross-generation comparison.

Table 11: Appendix: Comprehensive evaluation of legacy and reference foundation models on NVIDIA A100 (Pass@1 and Pass@5).

Method & Setup	Overall (210)		ATen (110)		vLLM (50)		cuBLAS (50)	
	Acc (%)	Spd (×)	Acc (%)	Spd (×)	Acc (%)	Spd (×)	Acc (%)	Spd (×)
LLM Sampling Methods								
Pass@1 (Opus-4.5)	35	0.78	33	0.94	22	0.83	54	0.59
Pass@1 (GPT-5.4)	21	0.69	21	0.76	30	0.54	12	0.65
Pass@1 (GPT-5.2)	21	0.55	24	0.53	22	0.64	12	0.57
Pass@1 (GLM-4.7)	12	0.56	15	0.49	10	1.30	10	0.35
Pass@5 (Opus-4.5)	50	0.73	53	0.91	28	0.88	64	0.46
Pass@5 (GPT-5.4)	35	0.72	41	0.87	38	0.44	20	0.60
Pass@5 (GPT-5.2)	38	0.52	47	0.49	36	0.58	18	0.56
Pass@5 (GLM-4.7)	23	0.67	26	0.55	22	1.10	20	0.70

I Fast_p Evaluation and Cost Results

This section reports the full fast_p distribution for all methods on NVIDIA A100 (Table 13), across six heterogeneous platforms (Table 14), and the complete cross-platform cost breakdown (Table 15).

Table 12: Appendix: fast_p evaluation of legacy and reference foundation models on NVIDIA A100.

Method & Setup	Overall (210)			ATen (110)			vLLM (50)			cuBLAS (50)		
	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$
LLM Sampling Methods												
Pass@1 (Opus-4.5)	20	16	2	25	22	2	16	14	6	10	4	0
Pass@1 (GPT-5.4)	12	9	0	17	12	1	8	6	0	6	4	0
Pass@1 (GPT-5.2)	9	7	1	15	10	2	6	6	2	0	0	0
Pass@1 (GLM-4.7)	7	5	1	8	5	1	10	10	4	0	0	0
Pass@5 (Opus-4.5)	30	21	4	43	31	3	20	14	8	12	6	2
Pass@5 (GPT-5.4)	21	13	1	33	21	1	10	6	2	8	4	0
Pass@5 (GPT-5.2)	18	11	1	28	18	1	8	6	4	4	0	0
Pass@5 (GLM-4.7)	12	8	3	15	7	1	14	12	8	4	4	2

Table 13: fast_p evaluation on NVIDIA A100 across 210 operators. The metric measures strict performance compliance ($\text{speedup} > 0.8\times, 1.0\times, 1.5\times$) across different generation paradigms and foundation models.

Method & Setup	Overall (210)			ATen (110)			vLLM (50)			cuBLAS (50)		
	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$
LLM Sampling Methods												
Pass@1 (Opus-4.6)	22	17	2	32	24	1	12	12	4	12	6	2
Pass@1 (GLM-5.0)	7	6	2	6	5	0	10	10	6	4	4	4
Pass@1 (Qwen3.5-27b)	5	3	0	7	5	0	2	2	2	4	0	0
Pass@1 (MiniMax M-2.5)	1	1	0	3	3	0	0	0	0	0	0	0
Pass@5 (Opus-4.6)	32	21	3	47	31	3	18	14	4	12	6	4
Pass@5 (GLM-5.0)	15	11	4	20	11	2	16	16	8	10	8	4
Pass@5 (Qwen3.5-27b)	9	7	2	12	10	1	8	6	6	4	0	0
Pass@5 (MiniMax M-2.5)	10	8	1	16	12	1	4	4	2	4	2	2
Vanilla Agentic Frameworks												
Claude Code (Opus-4.6)	51	35	6	69	44	4	48	40	14	14	10	4
Claude Code (GLM-5.0)	40	27	6	54	32	4	36	32	12	16	12	4
Claude Code (Qwen3.5-27b)	35	23	5	53	33	4	26	20	10	6	6	4
Claude Code (MiniMax M-2.5)	29	17	1	45	25	1	16	14	2	4	2	2
OpenCode (Opus-4.6)	51	33	5	74	47	5	34	22	4	20	14	6
OpenCode (GLM-5.0)	47	33	5	65	46	5	36	28	8	18	10	4
OpenCode (Qwen3.5-27b)	32	26	4	40	29	1	38	36	14	8	8	2
OpenCode (MiniMax M-2.5)	22	12	2	35	17	0	12	10	6	4	4	2
Kernel-Specialized Agents												
AKO4all (Opus-4.6)	65	47	10	85	59	6	56	50	20	30	16	10
CUDA Opt. Skill (MiniMax M-2.5)	28	18	4	42	25	4	16	10	4	6	0	0
AutoKernel (GLM-5.0)	60	38	4	87	53	0	34	32	12	22	10	6
AutoKernel (Qwen3.5-27b)	42	28	3	69	44	0	16	16	6	8	6	4
AutoKernel (MiniMax M-2.5)	39	24	2	66	37	0	18	18	8	2	2	2

Table 14: fast_p cross-platform evaluation on 110 ATen operators. Strict speedup compliance ($\text{fast}_{0.8}, \text{fast}_{1.0}, \text{fast}_{1.5}$) across six heterogeneous hardware ecosystems.

Method & Setup	NVIDIA			Platform A			Platform B			Platform C			Platform D			Platform E		
	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$	$\text{fast}_{0.8}$	$\text{fast}_{1.0}$	$\text{fast}_{1.5}$
LLM Sampling Methods																		
Pass@1 (Opus-4.6)	32	24	1	9	6	5	30	21	8	31	16	4	26	18	1	28	22	3
Pass@1 (Qwen3.5-27b)	7	6	0	1	1	0	3	1	0	6	4	0	6	3	0	5	3	0
Pass@1 (MiniMax M-2.5)	3	3	0	1	1	0	6	3	3	4	2	0	5	4	0	4	4	0
Pass@5 (Opus-4.6)	35	18	5	11	8	6	42	33	14	43	27	5	51	33	3	41	33	5
Pass@5 (Qwen3.5-27b)	12	10	1	4	2	0	10	7	3	13	11	0	10	2	0	14	7	0
Pass@5 (MiniMax M-2.5)	16	12	1	4	3	0	8	3	2	10	8	2	6	3	0	8	7	0
Vanilla Agentic Frameworks																		
Claude Code (Opus-4.6)	69	44	4	12	9	6	61	45	23	65	46	6	67	42	3	60	39	6
Claude Code (GLM-5.0)	40	27	6	9	6	6	47	35	22	41	24	2	40	26	0	44	30	4
Claude Code (Qwen3.5-27b)	53	33	4	16	9	7	43	31	12	58	47	11	61	39	5	19	11	2
Claude Code (MiniMax M-2.5)	46	46	1	8	7	6	46	36	16	44	31	3	51	25	2	46	31	5
Kernel-Specialized Agents																		
AKO4all (Opus-4.6)	83	57	6	16	12	7	81	55	8	79	56	10	82	60	10	75	48	7
CUDA Opt. Skill (MiniMax M-2.5)	42	25	4	9	7	5	45	29	13	41	27	4	43	23	1	38	24	3
AutoKernel (GLM-5.0)	87	53	0	47	25	0	56	28	1	57	29	2	59	28	0	26	21	0
AutoKernel (Qwen3.5-27b)	69	44	0	18	12	3	75	38	3	64	36	2	72	41	1	19	16	0
AutoKernel (MiniMax M-2.5)	66	37	0	44	26	2	70	53	15	64	35	0	70	35	1	48	37	0

Table 15: Cross-platform computational cost evaluation on 110 ATen operators (agentic methods only). Reports Total Tokens (M), Tokens per Success (M), and Total Execution Time (h) across six heterogeneous hardware ecosystems.

Method & Setup	NVIDIA			Platform A			Platform B			Platform C			Platform D			Platform E		
	Tok (M)	Tok/Suc	Time (h)	Tok (M)	Tok/Suc	Time (h)	Tok (M)	Tok/Suc	Time (h)	Tok (M)	Tok/Suc	Time (h)	Tok (M)	Tok/Suc	Time (h)	Tok (M)	Tok/Suc	Time (h)
Vanilla Agentic Frameworks																		
Claude Code (Opus-4.6)	84	0.83	9	173	1.77	18	128	1.25	16	107	1.11	15	111	1.05	15	123	1.35	16
Claude Code (GLM-5.0)	243	1.67	45	197	2.65	43	174	2.43	44	186	2.74	35	233	1.52	40	234	5.71	47
Claude Code (Qwen3.5-27b)	175	1.59	48	199	2.32	34	243	2.96	37	202	2.46	52	211	2.34	55	-	-	-
Claude Code (MiniMax M-2.5)	191	1.73	50	282	3.70	32	255	3.21	44	172	2.15	51	175	1.92	41	247	3.25	42
Kernel-Specialized Agents																		
AKO4all (Opus-4.6)	422	4.39	44	397	4.31	42	396	4.08	46	381	3.89	35	412	3.90	39	423	4.27	48
CUDA Opt. Skill (MiniMax M-2.5)	294	4.26	30	187	3.53	22	289	4.13	32	261	3.68	31	304	4.10	34	336	4.10	47
AutoKernel (GLM-5.0)	113	1.17	47	231	3.99	55	200	3.23	54	174	2.48	55	167	2.58	54	151	5.39	47
AutoKernel (Qwen3.5-27b)	133	1.74	36	136	3.17	55	178	2.15	51	131	1.82	54	141	1.74	50	166	7.22	53
AutoKernel (MiniMax M-2.5)	162	2.21	37	245	3.66	55	226	2.89	45	211	2.89	53	209	2.66	54	248	4.59	54

J Additional Analysis on Generation Behaviors

This section provides supplementary observations regarding model-specific generation bottlenecks and anti-hack interception statistics.

Generation truncation in GLM-5.0. Examining GLM-5.0’s generation logs under Pass@5, we find that a non-trivial fraction of generated kernels are incomplete: the model frequently produces excessive intermediate variables and verbose comments, causing core code logic to be truncated before completion within the uniform 16K max token limit. This generation inefficiency—low effective information per token—directly reduces the number of functionally correct submissions and explains the limited Pass@5 gain observed for GLM-5.0.

Anti-hack interception patterns. Table 16 shows a snapshot of the per-model anti-hack interception counts under Pass@1 and Pass@5. Hacks are heavily concentrated in ATen (29/30 interceptions under Pass@5), with vLLM accounting for almost zero. This is consistent with our findings in Section 4.2: vLLM’s non-standard API forces models to actually write complex logic, making simple shortcut strategies (like calling an existing high-level library function) unviable.

Table 16: Snapshot of anti-hack interception details (hack count and subset distribution).

Model	Pass@1		Pass@5	
	Hacks	Subset	Hacks	Subset
Opus-4.6	3	3 vLLM	7	7 ATen
Opus-4.5	3	2 ATen + 1 vLLM	2	1 ATen + 1 vLLM
GPT-5.4	3	3 ATen	8	8 ATen
GPT-5.2	3	3 ATen	5	5 ATen
GLM-5.0	1	1 ATen	1	1 ATen
GLM-4.7	0	—	7	7 ATen
Total	13	10 ATen + 3 vLLM	30	29 ATen + 1 vLLM

K Platform-Specific Failure Patterns

Universal Algorithmic Failures. The following bug patterns appear consistently across all platforms and models: (1) *Infinite dispatch recursion*: calling the overridden ATen operator internally triggers unbounded recursion—the most frequent single failure mode; (2) *Hallucinated Triton APIs*: models generate calls to non-existent functions such as `tl.pow`, `tl.einsum`, and `tl.gather`; (3) *Missing overload variants*: bare-name, `_out`, and `_dimname` variants required by the dispatch table are frequently omitted; (4) *Algorithmically hard operators*: `matmul`, `linear`, `sort`, and `cumsum` require cross-block parallel algorithms (matrix tiling, prefix scan, sorting networks) that agents rarely converge to within the iteration budget; `matmul` and `linear` additionally have 10+ overload variants with complex dispatch logic; (5) *BLOCK_SIZE conflicts*: `@triton.autotune` decorator definitions clash with manual `BLOCK_SIZE` assignments in the same kernel; (6) *Unsupported Python syntax*: Triton rejects `break` statements and chained boolean expressions that are valid Python; (7) *Non-contiguous memory*: operators like `as_strided` and `copy_` require non-contiguous mem-

ory semantics that Triton’s blocked access model cannot express; (8) *Stochastic API divergence*: bernoulli and poisson exhibit behavioral differences between Triton’s random API and PyTorch’s reference implementation.

Heterogeneous-Platform Failures. Alternative platforms exhibit additional failure modes beyond the type-system and IR compatibility issues noted in Section 4.5: (1) *Pointer/block type mismatch*: `tl.store(scalar_ptr, block_value)` is accepted by CUDA but rejected on certain alternative backends, which require pointer and value types to match; (2) *LLVM IR incompatibility*: certain Triton IR patterns trigger `PassManager::run` failed errors on some backends, affecting operators such as `fmax`, `scatter`, and `full` (31 observed occurrences on one platform); (3) *32-bit pointer addressing*: some platforms use 32-bit pointer arithmetic versus NVIDIA’s 64-bit, causing memory size too large to fit in 32 bit errors on large-tensor operators like `arange` and `argmax`; (4) *Missing math APIs*: `tl.acosh`, `tl.asin`, `tl.math.tanh`, `tl.language.math.pow`, and `tl.language.math.isinf` are absent on certain platforms and must be manually reimplemented using primitive operations; (5) *Incomplete BFloat16 support*: platforms with partial BFloat16 coverage silently fail on kernels that assume native BFloat16 computation without explicit type fallback; (6) *Runtime API failures*: certain backends raise runtime errors on memory-sensitive operators (`arange`, `copy_`, `index_put_`, `gather`) when stride layouts or data-type combinations fall outside supported configurations.

L Ablation Study: Per-Subset Breakdown

Table 17 provides the full per-subset ablation breakdown for the study presented in Section 4.7. Figure 6 visualizes the progressive trend across all four subsets.

Table 17: Full ablation study on Opus-4.6 (NVIDIA A100, 210 operators). Acc (%) is clean pass rate; Spd (×) is two-level geometric mean speedup.

Feedback	Mode	Iteration Method	Overall (210)		ATen (110)		vLLM (50)		cuBLAS (50)	
			Acc	Spd	Acc	Spd	Acc	Spd	Acc	Spd
No feedback	Pass@1 (Temp=0) Single-step Agent	Single greedy generation	41	0.70	39	0.90	20	0.76	68	0.49
		Single-step tool call	44	0.80	50	0.82	24	1.02	50	0.68
Text feedback	Pass@5 No-Reflect Pass@5 Reflection	Independent sampling, 5 rounds	57	0.68	62	0.79	28	0.71	74	0.49
		Fixed traceback-driven reflection	60	0.72	72	0.79	24	0.92	68	0.52
Execution feedback	Claude Code AKO4all	Autonomous interactive debugging	87	0.78	92	0.86	68	1.02	94	0.51
		Kernel-specialized agent	83	0.97	91	1.00	64	1.62	84	0.61

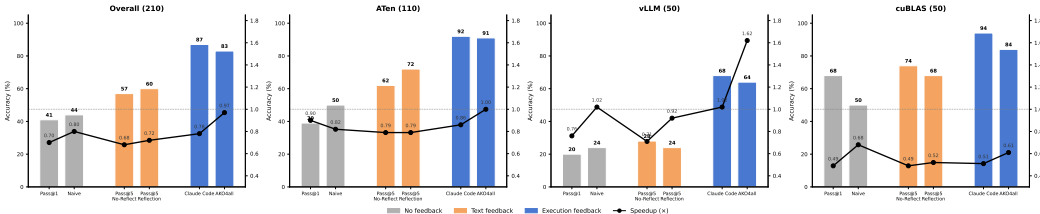


Figure 6: Per-subset ablation breakdown across four subsets (Overall, ATen, vLLM, cuBLAS). Bars show accuracy; black line shows speedup. The interactive debugging jump is consistent across all subsets, with the largest gain on vLLM (+44 pp).

M Reflection Mode Per-Operator Code Similarity Details

Table 18 compares the cross-round code similarity for 21 operators that both Reflection and No-Reflection modes fail to solve. Similarity is computed via line-level comparison using `difflib.SequenceMatcher`, with each operator averaged over 4 round transitions (round $i \rightarrow i+1$).

Table 18: Cross-round code similarity for 21 jointly failed operators under Reflection vs. No-Reflection modes.

Operator	Refl.	No-Refl.
_to_copy	86.4%	40.4%
add	90.5%	39.9%
affine_grid_generator	78.5%	43.1%
bernoulli	88.2%	28.6%
binary_cross_entropy_with_logits	84.5%	28.5%
cumsum	86.6%	43.7%
div	67.5%	26.4%
empty_strided	87.0%	52.8%
huber_loss	96.0%	25.1%
i0	90.6%	16.7%
linear	91.4%	58.7%
matmul	92.2%	34.8%
mean	97.1%	28.9%
mish_backward	94.7%	43.6%
mm	94.4%	58.2%
mul	97.0%	30.7%
poisson	87.3%	21.3%
pow	92.5%	36.0%
rrelu_with_noise	91.0%	45.0%
sort	71.2%	17.8%
sub	91.2%	42.3%
Overall Avg.	88.4%	36.3%

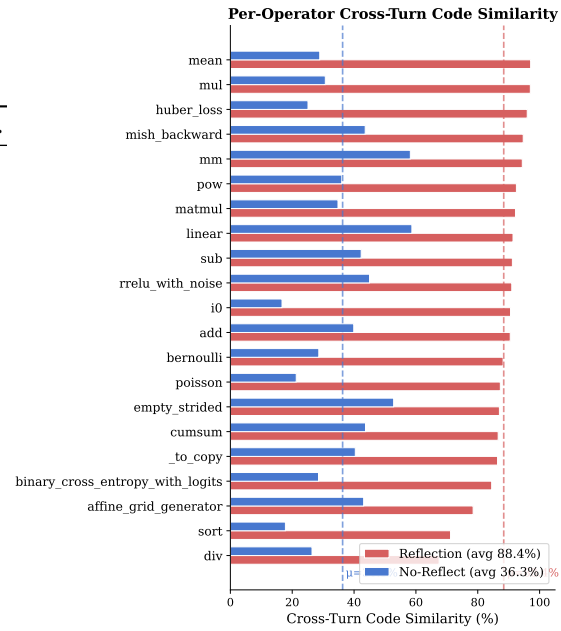


Figure 7: Cross-turn similarity for 21 jointly failed operators. Reflection clusters high (local patching); No-Reflect spreads wide (global search).